Beyond Development: Explaining Climate Resilience Across Indian Districts

A PROJECT REPORT

Submitted by

Sehjodh Grewal [RA2411031010081]
Devanshu Kumar [RA2411031010102]
Deepankar [RA2411031010103]
Tanmay Singh [RA2411031010118]

Under the Guidance of

Dr. BANU PRIYA P

Assistant Professor, Department of Networking and Communications

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Information Technology



DEPARTMENT OF NETWORKING AND COMMUNICATIONS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND
TECHNOLOGY KATTANKULATHUR-
603 203

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSC314P – Big Data Essentials mini-project report titled "**Beyond Development: Explaining Climate Resilience Across Indian Districts**" is the bonafide work of "Sehjodh Grewal [RA2411031010081], Devanshu Kumar [RA2411031010102], Deepankar [RA2411031010103], Tanmay Singh [RA2411031010118]" who carried out the mini-project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Panel Reviewer I

SIGNATURE

Dr. Banu Priya P
Assistant Professor
Department of Networking and Communications

SRM Institute of Science and Technology

Panel Reviewer II

SIGNATURE

Dr. Shanmuganathan V
Assistant Professor
Department of Networking and Communications
SRM Institute of Science and Technology

Department of Networking and Communications

SRM Institute of Science & Technology

Own Work* Declaration Form

Degree/ Course	**: B-Tech CSE-IT/ Big Data Essentials**
Student Names	**: Sehjodh Grewal, Devanshu Kumar, Deepankar, Tanmay Singh**
Registration Numbers	**: RA2411031010081, RA2411031010102, RA2411031010103, RA2411031010118**
Title of Work	**: Beyond Development: Explaining Climate Resilience Across Indian Districts**

I / We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism**, as listed in the University Website, Regulations, and the Education Committee guidelines.

I / We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:
I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above. Sehjodh Grewal Devanshu Kumar Deepankar Tanmay Singh (RA2411031010081) (RA2411031010102) (RA2411031010103) (RA2411031010118)
03/11/2026

ACKNOWLEDGEMENT

We would like to sincerely thank our faculty, Dr. P Banu Priya, from the Department of Networking and Communications at SRM Institute of Science and Technology, Chennai, for her invaluable guidance, consistent support and constructive feedback throughout the development of our project, “Beyond Development: Explaining Climate Resilience Across Indian Districts.” Her expertise in big data analytics and machine learning gave us a strong foundation to approach this problem with confidence and care.

We also appreciate the assistance and encouragement of the faculty members of the School of Computing, whose dedication to a collaborative and innovative learning environment enriched our experience, and our classmates for their cooperation and exchange of ideas throughout the project.

We are grateful to the organisations whose open data made this work possible: the India Meteorological Department for the gridded rainfall and temperature data, NITI Aayog’s National Data and Analytics Platform for the district climate vulnerability indicators, the Office of the Registrar General and Census Commissioner for the Census 2011 Primary Census Abstract, and the DataMeet community for the district boundaries. We also thank the developers and maintainers of Apache Hadoop, Hive and Spark, PyTorch, scikit-learn and pandas, and Google Colab for providing the computing environment.

Finally, we extend our deepest gratitude to our families for their unwavering support and motivation. Through this project we hope to contribute, in our own way, to a better understanding of why some districts cope with climate stress better than others, and to show how big data tools and deep learning can support evidence-based adaptation planning.

ABSTRACT

Development indicators such as literacy and the share of farm workers are often used as a shorthand for how well an Indian district can cope with climate stress. The data only partly support this: development explains just 13% of the variation in the 2021 district climate vulnerability index, and 181 of 469 districts (38.6%) break the expected pattern, being developed yet vulnerable or less developed yet resilient. This project explains these anomalies by combining the NITI Aayog vulnerability index, Census 2011 development variables and thirty years (1991–2020) of daily India Meteorological Department rainfall and temperature grids, about 212 million gridded values. A single-node Hadoop pipeline stores the data in HDFS (1.08 GB), converts 7,013,120 district-day rows into 19,200 district-year climate indices with MapReduce on YARN, builds the 30-year baseline and the 469-district analysis table with Hive, and groups districts into climate regimes with Spark k-means. Preprocessing uses no imputation and is verified with eight error metrics; area-weighted aggregation avoids a rainfall error of 287 mm (RMSE) that a naive nearest-cell method would introduce. On this table we propose B-ACPN, a Boosted Attentive Climate Push Network, in which a random-forest stage gives a first risk score and a small neural network adds a development term and a development-gated, attention-weighted climate push, averaged over five networks, with Monte Carlo Dropout and conformal uncertainty. On 25 repeated cross-validation folds B-ACPN reaches a ROC-AUC of 0.820, accuracy of 0.738 and F1-score of 0.728, the highest of six models; it is significantly better than random forest, a standard MLP, a linear SVM and logistic regression and comparable to gradient-boosted trees ($p = 0.55$). Its climate push separates the anomalous districts from the expected ones ($p < 0.001$), showing that climate, not only development, explains why developed districts can remain vulnerable.

TABLE OF CONTENTS

CHAPTER NO.	TOPIC	PAGE NO.
	ABSTRACT	**iii**
	LIST OF FIGURES	**vi**
	LIST OF TABLES	**vii**
1	**INTRODUCTION**	**11**
	1.1 Problem Statement	**11**
	1.2 Motivation	**11**
	1.3 Objectives	**11**
	1.4 Ideas	**12**
	1.5 Solution Proposed	**12**
	1.6 Challenges	**12**
2	**LITERATURE REVIEW**	**13**
3	**PROPOSED METHODOLOGY**	**14**
	3.1 Data Collection	**14**
	3.2 Data Preprocessing	**14**
	3.3 Feature Engineering	**15**
	3.4 Model Architecture	**15**
	3.5 Evaluation Metrics	**16**
	3.6 Visualization Techniques	**16**
4	**IMPLEMENTATION**	**17**
	4.1 Data Acquisition and Loading	**17**
	4.2 Data Cleaning and Preprocessing	**18**
	4.3 MapReduce on YARN	**18**
	4.4 Hive Baseline and Join	**19**
	4.5 Spark Climate Regimes	**19**
	4.6 B-ACPN Training	**20**
	4.7 Comparison Models and Evaluation Protocol	**21**
	4.8 System Summary	**21**

5 RESULTS AND DISCUSSIONS 22

5.1 Preprocessing Results **22**

5.2 Comparison with Existing Models **23**

5.3 Effect of Preprocessing on the Models **25**

5.4 Ablation Study **26**

5.5 Climate-Push Analysis **27**

5.6 Summary of Findings **28**

6 CONCLUSION AND FUTURE WORK 29

7 REFERENCES 30

LIST OF FIGURES

Figure No	Title	Page No
3.1	Architecture of the Proposed B-ACPN Model	**16**
4.1	Complete Workflow of the Project	**17**
4.2	HDFS Storage by Zone	**18**
4.3	MapReduce Runtime for 1, 10 and 30 Years (Single Node)	**19**
4.4	Spark k-means Silhouette Score by Number of Clusters	**20**
4.5	B-ACPN Training and Validation Loss (Early Stopping)	**21**
5.1	Data-Quality Problems Before vs After Preprocessing	**23**
5.2	District Retention Through Preprocessing (No Imputation)	**23**
5.3	ROC and Precision–Recall Curves of the Six Models	**25**
5.4	Confusion Matrices Summed over the 25 Test Folds	**25**
5.5	Accuracy, F1, ROC-AUC and RMSE Before vs After Preprocessing	**26**
5.6	Ablation: One B-ACPN Component Removed at a Time	**27**
5.7	Mean Feature-Attention Weight of Each Climate Variable	**28**
5.8	Climate Push of Every District	**28**

LIST OF TABLES

Table No	Title	Page No
3.1	Data Sources Used in the Project	**14**
3.2	Features Used by the Models	**15**
4.1	HDFS Zones and Storage	**17**
4.2	Summary of the Big Data Jobs	**20**
4.3	B-ACPN Model Specification	**21**
5.1	Preprocessing Checks Before vs After	**22**
5.2	Preprocessing Performance Metrics	**23**
5.3	Comparison with Existing Models (25 Folds)	**24**
5.4	Detailed Classification Metrics	**24**
5.5	Classification Metrics Before → After Preprocessing	**26**
5.6	Ablation of B-ACPN (ROC-AUC, 25 Folds)	**27**
5.7	Mean Climate Push by Group	**27**

LIST OF ABBREVIATIONS

Abbreviation	Full Form
B-ACPN	Boosted Attentive Climate Push Network
CV	Cross-Validation
ECE	Expected Calibration Error
GBT	Gradient-Boosted Trees
HDFS	Hadoop Distributed File System
IMD	India Meteorological Department
LOSO	Leave-One-State-Out
LR	Logistic Regression
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MC	Monte Carlo
MCC	Matthews Correlation Coefficient
MedAE	Median Absolute Error
MLP	Multilayer Perceptron
MSE	Mean Squared Error
NDAP	National Data and Analytics Platform
NRMSE	Normalised Root Mean Square Error
PR-AUC	Area Under the Precision–Recall Curve
RF	Random Forest
RMSE	Root Mean Square Error
ROC-AUC	Area Under the ROC Curve
SVM	Support Vector Machine
YARN	Yet Another Resource Negotiator

CHAPTER 1

INTRODUCTION

India is among the countries most exposed to heavy rainfall, droughts and heat waves, and district administrations are the level at which climate adaptation is planned. National assessments rank districts by climate vulnerability, and a common assumption is that more developed districts, with higher literacy and less dependence on farming, are better able to cope. This project tests that assumption with thirty years of daily climate data and explains, for every district, how much of its vulnerability comes from climate rather than from development.

1.1 Problem Statement

Splitting the 469 districts of our sample at the median of a development score and at the median of the vulnerability index gives four groups. 143 districts are developed and have low vulnerability and 145 are less developed and highly vulnerable, as expected. But 92 districts are developed yet vulnerable (Anomaly A) and 89 are less developed yet resilient (Anomaly B). Development alone explains only 0.127 of the variance of the index (R^2). The research question is therefore: why do some developed Indian districts remain highly climate-vulnerable, while some less-developed districts are resilient?

1.2 Motivation

Answering this question needs long, fine-grained climate records. Thirty years of daily IMD grids for the whole country contain about 212 million values, and the same processing steps (aggregating days into years, years into a long-term baseline, and joining the result with development data) recur whenever the period or the variables change. This makes the problem a natural fit for a big data pipeline. At the same time, a standard classifier only says whether a district is highly vulnerable, not why. Planners need a model that is accurate and that also separates the climate contribution from the development contribution for every district.

1.3 Objectives

The specific objectives of the project are:

- Build a reproducible big data pipeline over 30 years (1991–2020) of daily IMD climate data using HDFS, MapReduce on YARN, Hive and Spark.
- Prepare a clean district-level dataset without imputation and verify every

preprocessing step with error metrics.

- Design a novel deep-learning model, B-ACPN, that predicts vulnerability and gives a per-district climate push with uncertainty.
- Compare B-ACPN fairly with five existing machine-learning models on identical folds using many metrics, confidence intervals and significance tests.
- Explain each district as climate-driven or development-driven.

1.4 Ideas

The core idea is to treat a district's risk as two parts: a baseline that development (and a strong tree model) can explain, and a climate push that raises or lowers the risk beyond that baseline. If the climate push is large and positive for developed-but-vulnerable districts and negative for less-developed-but-resilient ones, then climate explains the anomalies.

1.5 Solution Proposed

Raw sources are stored in HDFS; IMD grids are converted to district-day rows using area weights; MapReduce computes yearly climate indices; Hive averages them over 30 years and joins them with development data; Spark finds climate regimes and trains four comparison models. The proposed model B-ACPN combines a random-forest stage with a development-gated attention network and an ensemble of five networks, and reports a climate push, the most important climate factor and an uncertainty for every district.

1.6 Challenges

The main challenges were: matching district names between the 2021 NDAP data and Census 2011, where many districts were created or renamed after 2011; a very sparse NDAP table (75% of cells empty); converting coarse climate grids to districts without bias; running Hadoop, Hive and Spark inside Google Colab, which has no SSH and wipes its disk every session; and building a model that is both the most accurate and explainable on only 469 districts.

CHAPTER 2

LITERATURE REVIEW

Climate vulnerability in India has been mapped at district level by Mohanty and Wadhawan [1], who combined exposure to hydro-meteorological extremes with sensitivity and adaptive capacity, and by the Department of Science and Technology [2], which published a common framework for district vulnerability assessment. The IPCC [3] stresses that vulnerability depends on both exposure and adaptive capacity. These studies build composite indices but do not ask why districts with similar development end up with different vulnerability, which is the focus of this project.

Long gridded climate records are the basis of such analysis. Pai et al. [4] developed the IMD daily rainfall dataset at 0.25° resolution, and Srivastava et al. [5] the IMD daily temperature dataset at 1°. Both are distributed as yearly binary files, which makes 30-year, all-India processing a data-intensive task.

On the big data side, the Hadoop Distributed File System [6] stores large files as replicated blocks, MapReduce [7] expresses batch aggregations as map and reduce functions, and YARN [8] schedules them. Hive [9] adds SQL over HDFS for aggregation and joins, while Apache Spark [10] keeps data in memory, which suits iterative algorithms; its MLlib library [11] provides standard classifiers. We use each tool for the step it fits best.

For small tabular datasets, tree ensembles such as random forests [12] and gradient boosting [13] remain strong baselines, and Grinsztajn et al. [14] showed that they still often outperform deep networks. Attention over input features, as in TabNet [15], makes a network's use of features visible, stacking [16] feeds one model's out-of-fold predictions to another, and residual connections [17] ease the training of deeper networks. B-ACPN combines these ideas so that the trees provide accuracy and the network adds a structured, explainable correction.

For uncertainty, Monte Carlo Dropout [18] approximates Bayesian uncertainty by keeping dropout active at prediction time, and split conformal prediction [19] turns any classifier's scores into prediction sets with a coverage guarantee. For a fair comparison of models evaluated with repeated cross-validation, Nadeau and Bengio [20] proposed a corrected resampled t-test, and the silhouette score [21] is a standard way to choose the number of clusters. Together, the literature shows the pieces exist, but not a pipeline and model that explain district climate anomalies in India.

CHAPTER 3

PROPOSED METHODOLOGY

This chapter describes the data, the preprocessing, the features, the proposed model B-ACPN, the evaluation metrics and the visualisation approach. The overall flow is: collect the sources, clean and match them, process the climate data with big data tools, train B-ACPN and five existing models on identical folds, and explain each district with its climate push.

3.1 Data Collection

Table 3.1 lists the five public sources. The outcome is the 2021 district climate vulnerability index from NITI Aayog's National Data and Analytics Platform (NDAP, dataset 7160), split at its median into High and Low vulnerability. The IMD files are downloaded year by year in Google Colab to Google Drive, and each file is checked against its exact expected size before use.

Table 3.1: Data Sources Used in the Project

Dataset	Source	Year	Size / Format	Role
Climate Vulnerability Index	NITI Aayog NDAP 7160	2021	699 × 79, CSV	Target + 2 development variables
Primary Census Abstract	Census of India	2011	640 districts, CSV	3 development variables
Daily rainfall (0.25°)	IMD Pune	1991–2020	763 MB, binary	Climate hazards
Daily Tmax / Tmin (1°)	IMD Pune	1991–2020	84 MB, binary	Climate hazards
District boundaries	DataMeet	2011	641 polygons	Grid → district weights

3.2 Data Preprocessing

Census counts are converted to rates (literacy, female literacy and agricultural-worker share). NDAP and Census districts are matched inside state clusters (Andhra Pradesh with Telangana, Jammu and Kashmir with Ladakh) by exact name, an alias table and fuzzy matching; 602 of 699 NDAP districts are matched with no duplicate codes, and two wrong fuzzy matches (Sangareddy → Rangareddy and South West Garo Hills → South Garo Hills) are blocked. Because NDAP is very sparse, only well-covered variables are kept and districts with

missing values are dropped instead of imputed, leaving 469 complete districts in 21 states.

Grid values are converted to district values with area weights: each grid cell contributes in proportion to the share of the cell inside the district, multiplied by the cosine of its latitude, and the weights of each district sum to one. IMD missing codes stay missing and are never set to zero. An automatic orientation check confirms that Chennai's rainfall peaks in October–December. Features are standardised on the training rows of each fold only, so no test information leaks into training.

3.3 Feature Engineering

Thirteen features are used, five for development and eight for climate hazard (Table 3.2). The climate features are computed from the daily data by MapReduce and Hive and summarised over 1991–2020.

Table 3.2: Features Used by the Models

Group	Feature	Definition
Development	Literacy, female literacy	Literates / population aged 7+ (Census 2011)
Development	Agricultural-worker share	Cultivators + agricultural labourers / all workers
Development	MGNREGA days, yield variability	NDAP 2021 indicators
Climate	Annual rainfall, rain variability	30-year mean and year-to-year SD of annual rain
Climate	Max 5-day rain, heavy-rain days	Wettest 5 days; days with rain ≥ 64.5 mm
Climate	Longest dry spell	Longest run of days with rain < 1 mm
Climate	Mean Tmax, hot days, tropical nights	Mean daily maximum; days Tmax ≥ 35 °C; nights Tmin ≥ 25 °C

3.4 Model Architecture

The proposed model, B-ACPN (Boosted Attentive Climate Push Network), is shown in Fig. 3.1. It has three stages. In the tree stage, a random forest with 500 trees gives a first risk score from all 13 features; for training districts this score is computed out-of-fold over five inner folds, so the next stage never sees a score made on its own training rows. In the deep stage, a small neural network starts from that score and adds a development term (a linear layer on the 5 development features) and a climate push: the 8 hazard features pass through feature attention, a linear layer with ReLU and dropout, and a residual block, and the result is multiplied by a development gate between 0 and 2. Finally, five such networks with different seeds are averaged.

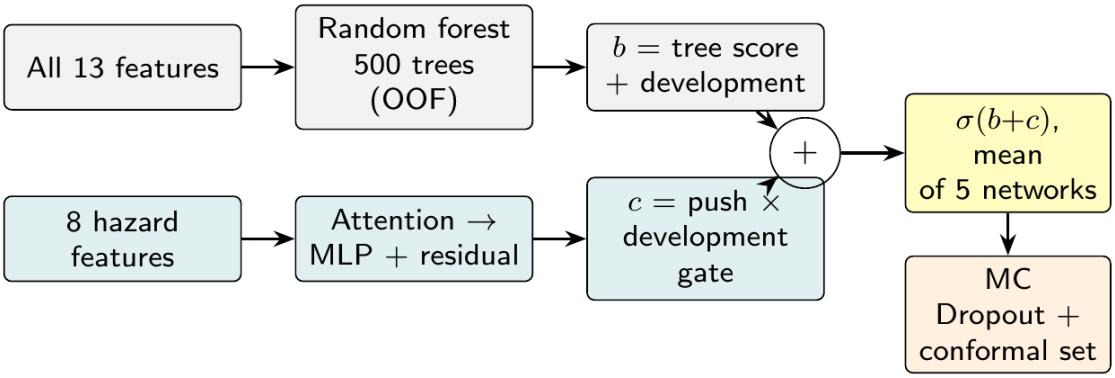


Fig. 3.1: Architecture of the Proposed B-ACPN Model

The risk of district i is σ(tree score + development term + climate push × gate), averaged over the five networks. A positive climate push means that climate adds risk beyond what the trees and development explain; a negative push means that climate protects the district. Attention shows which climate factor drives the push, Monte Carlo Dropout (50 passes) gives the uncertainty of the push, and split conformal prediction gives prediction sets with a 90% coverage target.

3.5 Evaluation Metrics

Preprocessing is evaluated with eight error metrics against an exact reference: RMSE, MSE, MAE, MAPE, R^2, NRMSE, median absolute error and bias. The classifiers are evaluated with ten classification metrics (accuracy, precision, recall, F1-score, specificity, ROC-AUC, log loss, MCC, Cohen's kappa and balanced accuracy) plus PR-AUC, Brier score, expected calibration error and RMSE of the predicted probability. All models use the same 25 folds (five-fold cross-validation repeated five times) and leave-one-state-out testing; differences are tested with the corrected resampled t-test [20].

3.6 Visualization Techniques

All figures are generated with Matplotlib directly from the results files, only after every table exists. They include before/after charts of the preprocessing, HDFS storage and MapReduce scaling, ROC and precision–recall curves, confusion matrices, ablation bars, the attention weights, the climate push of every district by group and an India map of the climate push.

CHAPTER 4

IMPLEMENTATION

The system is implemented as a Google Colab notebook backed by a GitHub repository. Colab runs a single-node, pseudo-distributed Hadoop 3.3.6 cluster (HDFS replication 1, 128 MB blocks, 2 vCPU, 12.7 GB RAM) with Java 8, Hive 3.1.3 and Spark 3.5.3. Fig. 4.1 shows the complete workflow. Because the cluster has one node, no multi-machine speed-up is claimed.

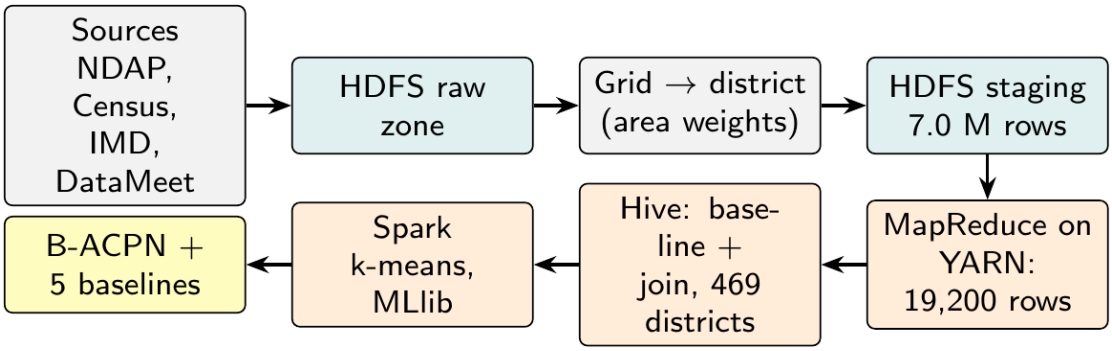


Fig. 4.1: Complete Workflow of the Project

4.1 Data Acquisition and Loading

The IMD files are downloaded to Google Drive (Colab wipes its local disk), and all sources are loaded into three HDFS zones with hdfs dfs -put (Table 4.1): raw for the untouched sources, staging for the district-day table and processed for the job outputs. Colab has no SSH, so the four Hadoop daemons (NameNode, DataNode, ResourceManager, NodeManager) are started directly. The storage measured with hdfs dfs -du is shown in Fig. 4.2; in total 1.08 GB are stored.

Table 4.1: HDFS Zones and Storage

HDFS zone	Contents	Size
/project/raw/imd/rain	30 yearly rainfall files	763.3 MB
/project/raw/imd/tmax, tmin	60 temperature files	84.2 MB
/project/raw (other)	NDAP, Census, boundaries	11.5 MB
/project/staging/district_day	7,013,120 district-day rows	222.8 MB
/project/processed	MapReduce, Hive and Spark outputs	1.3 MB

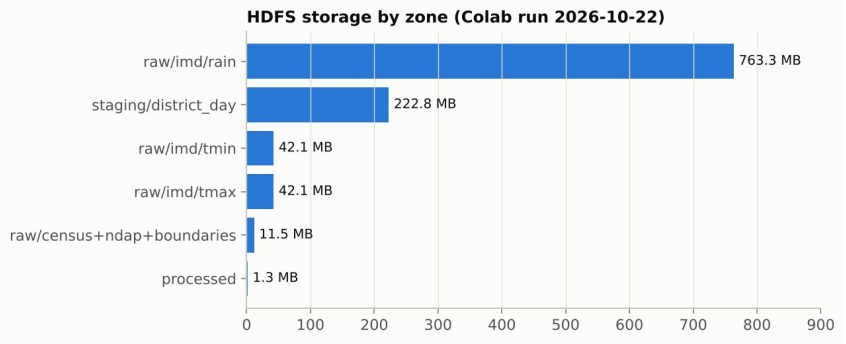


Fig. 4.2: HDFS Storage by Zone

4.2 Data Cleaning and Preprocessing

The preprocessing scripts compute the Census rates, the NDAP–Census crosswalk, the development table and the grid-to-district area weights (10,291 rainfall and 2,074 temperature cell–district pairs). The district-day table is then built as a weighted mean over the cells with data on each day, giving 7,013,120 rows with 0.62% of rainfall values missing, which stay missing.

4.3 MapReduce on YARN

A Hadoop Streaming job with a Python mapper and reducer turns the district-day rows into yearly climate indices. The mapper keys each row by (district, year); the reducer sorts the days of one district-year by date and computes seven indices: annual rainfall, heavy-rain days, maximum five-day rainfall, longest dry spell, mean daily maximum temperature, hot days and tropical nights. Incomplete years are left blank rather than set to zero. The job reads 7,013,120 rows and writes 19,200 district-year rows; on the single node it took 55, 140 and 333 seconds for 1, 10 and 30 years of input (Fig. 4.3). An independent pandas recomputation matches the MapReduce output to rounding error.

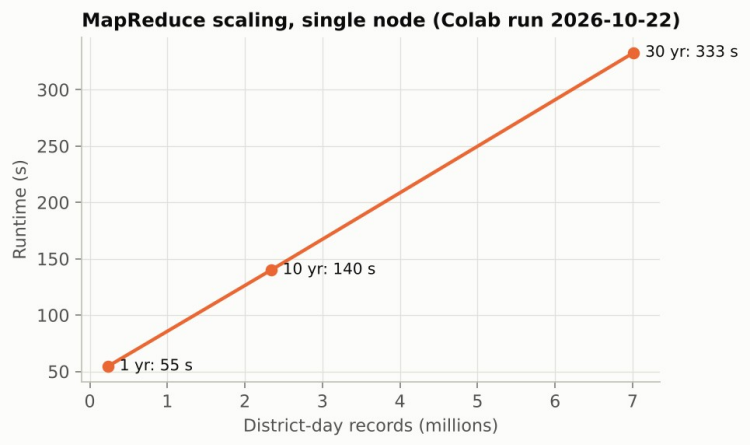


Fig. 4.3: MapReduce Runtime for 1, 10 and 30 Years (Single Node)

4.4 Hive Baseline and Join

A HiveQL script averages every index over 1991–2020 for each district, adds the year-to-year standard deviation of annual rainfall as rain variability, and joins the result with the development table on the Census district code. This produces the 469-district analysis table with 13 features and the target. The Hive output is identical to a pandas implementation (maximum difference 0.0).

4.5 Spark Climate Regimes

Spark reads the analysis table from HDFS and runs k-means on the eight standardised hazard variables for k = 2 to 8. The silhouette score is highest for k = 2 (0.651), giving two climate regimes of 374 and 95 districts (Fig. 4.4). The regimes are used later as an independent check of the climate push. Spark MLlib is also used to train four of the comparison models.

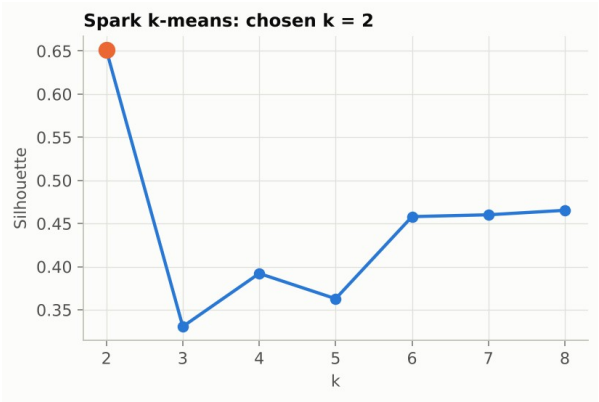


Fig. 4.4: Spark k-means Silhouette Score by Number of Clusters

Table 4.2: Summary of the Big Data Jobs

Job	Input	Output	Result
HDFS load	Raw + staging data	3 zones	1.08 GB
MapReduce on YARN	7,013,120 rows	19,200 district-years	333 s (30 years)
Hive	19,200 rows + development table	469-district table	Identical to pandas
Spark k-means	8 hazard variables	2 climate regimes	Silhouette 0.651

4.6 B-ACPN Training

Each B-ACPN network is trained in PyTorch with a class-weighted binary cross-entropy loss and the AdamW optimiser (learning rate 3×10^{-3}, weight decay 10^{-3}, batch size 64) for at most 300 epochs, with early stopping after 30 epochs without improvement on a stratified 15% validation split (Fig. 4.5). Each network has 780 trainable parameters in addition to the 500-tree forest, and one fold trains in 11.8 s on a CPU. The settings are summarised in Table 4.3.

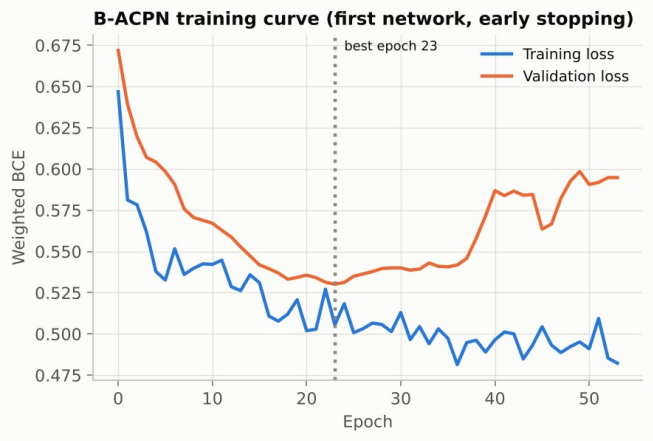


Fig. 4.5: B-ACPN Training and Validation Loss (Early Stopping)

Table 4.3: B-ACPN Model Specification

Component	Specification
Tree stage	Random forest, 500 trees, min leaf 2; out-of-fold score (5 inner folds)
Deep stage	Tree score + Linear(5 → 1) development term
Climate push	Attention (8) → Linear(8 → 16), ReLU, Dropout 0.3 → residual block → Linear(16 → 1)
Development gate	2σ(Linear(5 → 4), tanh, Linear(4 → 1))
Ensemble	5 networks (seeds 42, 1, 2, 3, 4), probabilities averaged
Uncertainty	MC Dropout (50 passes); conformal coverage 92.9% (target 90%)
Size / time	780 parameters per network; 11.8 s per fold

4.7 Comparison Models and Evaluation Protocol

Five existing models are trained on exactly the same folds: logistic regression, random forest (200 trees), gradient-boosted trees (100 iterations, depth 3) and a linear SVM in Spark MLlib, and a standard multilayer perceptron (13–64–32–16–1) in PyTorch. The folds are five-fold stratified cross-validation repeated five times (25 splits, seed 42) plus leave-one-state-out over the 21 states. Every model is also trained on the raw, unscaled features to measure the effect of preprocessing.

4.8 System Summary

The whole system runs from the Colab notebook: setup and data (sections 0–2), HDFS, MapReduce, Hive and Spark (3–7), B-ACPN (8), the comparison and tables (9–10) and the figures (11). Every result is saved to the repository, and the slides and this report are built from the same results.

CHAPTER 5

RESULTS AND DISCUSSIONS

This chapter presents the results of the preprocessing, the comparison of B-ACPN with the five existing models, the effect of preprocessing on the models, the ablation study and the climate-push analysis that answers the research question.

5.1 Preprocessing Results

Table 5.1 checks each preprocessing step against a reference, before and after. The Census population matches the official figure exactly, national literacy differs by only 0.0019 percentage points, the grid weights sum to one, no value is zero-filled, and the final table has no missing, duplicate or out-of-range values.

Table 5.1: Preprocessing Checks Before vs After

Check	Before / reference	After / measured	Error
Census population	official 1,210,854,977	1,210,854,977	0 (0.000%)
National literacy	official 72.98%	72.9819%	0.0019 pp
NDAP missing cells	75.0% of all cells	0 missing in 13 features	No imputation
District matching	699 names; 2 wrong fuzzy	602 matched, 0 duplicates	2 errors blocked
Grid weights Σw = 1	10,291 / 2,074 pairs	max |Σw−1| 7e-08	RMSE 1.2e-08
Final table	—	0 duplicates, 0 out of range	66 outliers kept

Table 5.2 reports the eight error metrics for the two steps with an exact reference. Replacing our area-weighted rainfall with the nearest grid cell would cause an RMSE of 287 mm and a MAPE of 11.1% per district (R^2 only 0.904); this is the error our method avoids. The MapReduce indices match an independent pandas recomputation almost perfectly.

Table 5.2: Preprocessing Performance Metrics

Metric	Full form	Nearest cell vs ours (mm)	MapReduce vs pandas
RMSE	Root Mean Square Error	286.9	0.003
MSE	Mean Squared Error	82,313.7	8.4e-06
MAE	Mean Absolute Error	151.7	0.003
MAPE	Mean Absolute Percentage Error	11.08%	0.008%
R^2	Coefficient of Determination	0.9042	0.999999
NRMSE	Normalised RMSE	0.034	1.9e-04
MedAE	Median Absolute Error	87.0	0.002
Bias / ME	Mean Error	5.045	-2.2e-05

Fig. 5.1 shows the data-quality problems before and after preprocessing and the feature scales before and after standardisation. Fig. 5.2 shows how many districts survive each stage and the signal each feature carries on its own; the rainfall signal-to-noise ratio is 3.98.

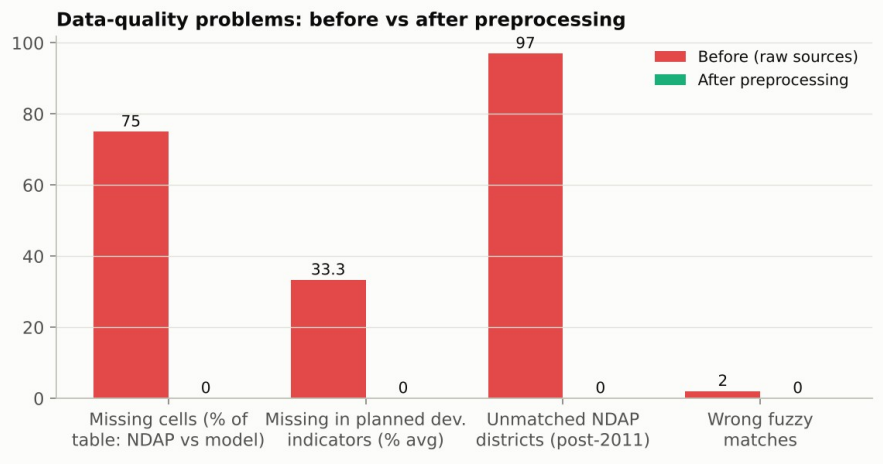


Fig. 5.1: Data-Quality Problems Before vs After Preprocessing

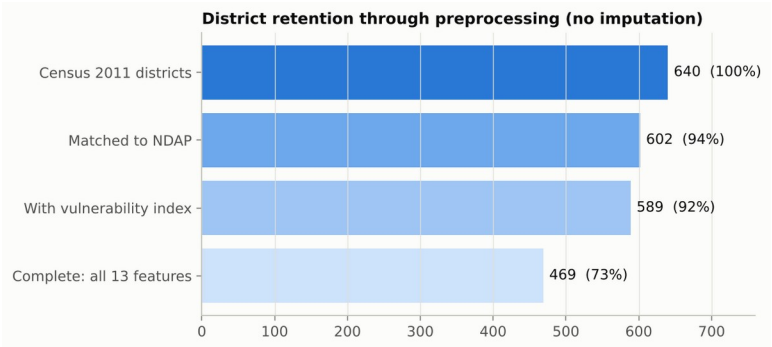


Fig. 5.2: District Retention Through Preprocessing (No Imputation)

5.2 Comparison with Existing Models

Table 5.3 compares B-ACPN with the five existing models on the same 25 folds. B-ACPN has the highest ROC-AUC (0.820), balanced accuracy, accuracy (0.738), F1-score

(0.728) and leave-one-state-out AUC (0.717). It is significantly better than RF, MLP, Linear SVM and LR ($p < 0.05$); its lead over GBT is smaller than one standard deviation and not significant ($p = 0.55$), so the two are reported as comparable.

Table 5.3: Comparison with Existing Models (25 Folds)

Model	ROC-AUC	Bal. acc.	Accuracy	F1	LOSO AUC	p vs B-ACPN
B-ACPN	0.820 ± 0.035	0.739	0.738	0.728	0.717	—
GBT	0.811 ± 0.034	0.726	0.725	0.724	0.698	0.550
RF	0.804 ± 0.036	0.715	0.715	0.708	0.706	0.042
MLP	0.784 ± 0.037	0.699	0.699	0.692	0.710	0.019
Linear SVM	0.755 ± 0.039	0.696	0.696	0.708	0.691	0.006
LR	0.747 ± 0.042	0.691	0.692	0.701	0.661	0.004

Table 5.4 lists further metrics. B-ACPN is best on precision, specificity, MCC, kappa, PR-AUC, log loss and Brier score. It is not best on recall, where the linear SVM is higher, or on calibration error, where the random forest is lower: B-ACPN is more careful about calling a district highly vulnerable.

Table 5.4: Detailed Classification Metrics

Model	Prec.	Recall	Spec.	MCC	Kappa	PR-AUC	Log loss	Brier	ECE
B-ACPN	0.765	0.700	0.778	0.482	0.477	0.844	0.516	0.174	0.108
GBT	0.736	0.718	0.733	0.454	0.451	0.826	0.570	0.188	0.131
RF	0.735	0.689	0.742	0.434	0.430	0.830	0.539	0.182	0.095
MLP	0.719	0.674	0.724	0.404	0.398	0.809	0.579	0.194	0.114
Linear SVM	0.689	0.733	0.659	0.396	0.392	0.751	n/a	n/a	n/a
LR	0.688	0.718	0.665	0.386	0.383	0.746	0.611	0.206	0.101

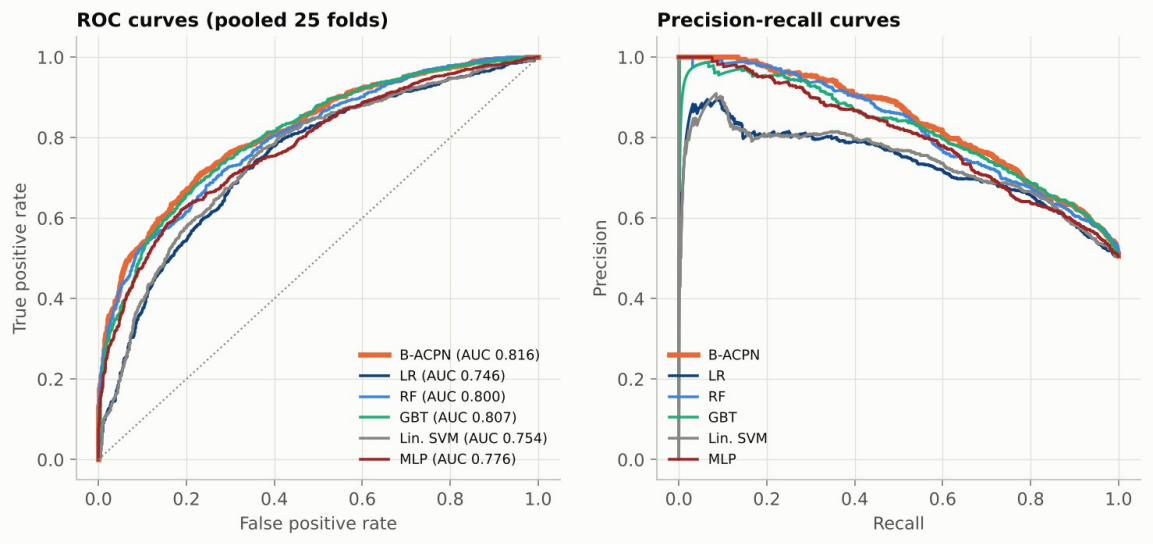


Fig. 5.3: ROC and Precision–Recall Curves of the Six Models

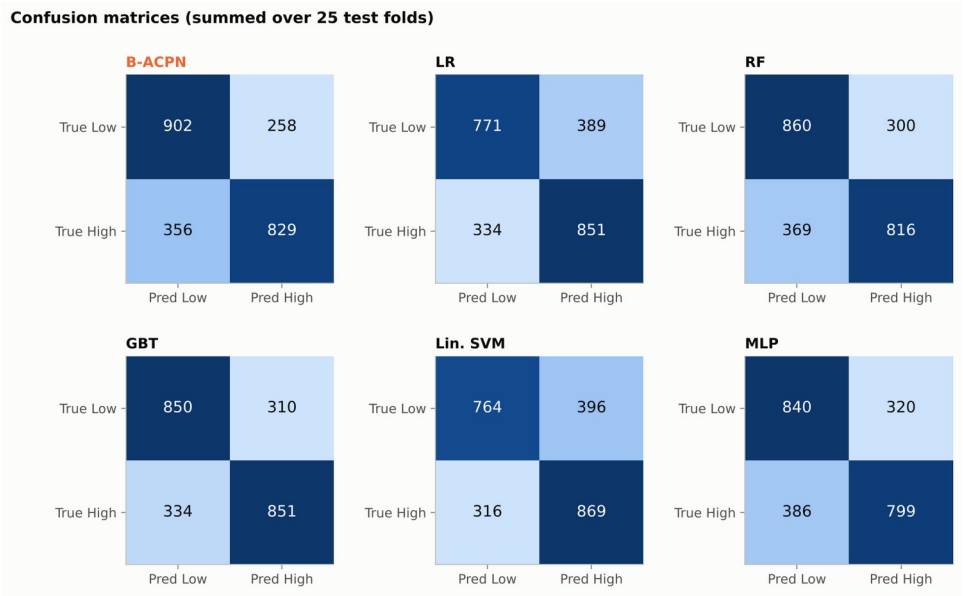


Fig. 5.4: Confusion Matrices Summed over the 25 Test Folds

5.3 Effect of Preprocessing on the Models

Every model was also trained on the raw, unscaled features. Table 5.5 shows the metrics before and after preprocessing. B-ACPN improves on all of them, for example ROC-AUC from 0.806 to 0.820. The MLP gains recall, F1 and ROC-AUC but loses precision and specificity, because after scaling it predicts High more often. Tree models ignore feature scale and Spark's linear models standardise internally, so they do not change (Fig. 5.5).

Table 5.5: Classification Metrics Before → After Preprocessing

Metric	B-ACPN	MLP	RF	GBT
Accuracy	0.735 → 0.738	0.711 → 0.699	0.715 → 0.715	0.726 → 0.725
Precision	0.760 → 0.765	0.776 → 0.719	0.735 → 0.735	0.737 → 0.736
Recall	0.700 → 0.700	0.611 → 0.674	0.689 → 0.689	0.718 → 0.718
F1-score	0.726 → 0.728	0.678 → 0.692	0.708 → 0.708	0.724 → 0.724
Specificity	0.770 → 0.778	0.813 → 0.724	0.742 → 0.742	0.734 → 0.733
ROC-AUC	0.806 → 0.820	0.759 → 0.784	0.805 → 0.804	0.811 → 0.811
Log loss ↓	0.528 → 0.516	0.588 → 0.579	0.539 → 0.539	0.570 → 0.570
MCC	0.474 → 0.482	0.437 → 0.404	0.434 → 0.434	0.455 → 0.454
Kappa	0.470 → 0.477	0.423 → 0.398	0.430 → 0.430	0.452 → 0.451
Balanced acc.	0.735 → 0.739	0.712 → 0.699	0.715 → 0.715	0.726 → 0.726
RMSE ↓	0.422 → 0.417	0.445 → 0.440	0.427 → 0.427	0.432 → 0.433

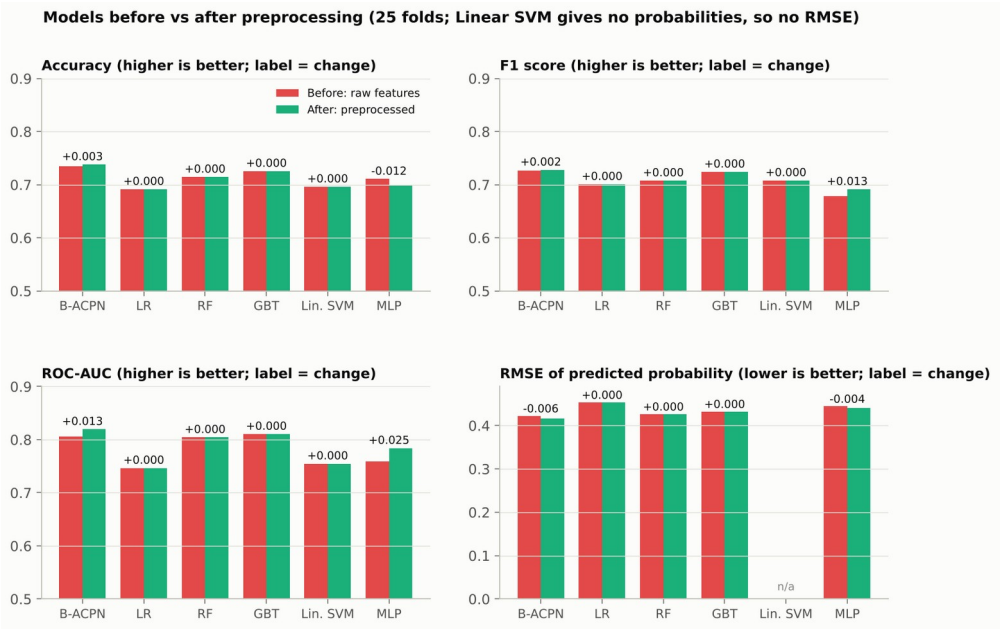


Fig. 5.5: Accuracy, F1, ROC-AUC and RMSE Before vs After Preprocessing

5.4 Ablation Study

Table 5.6 removes one part of B-ACPN at a time on the same folds. Removing the tree stage or the ensemble costs about 0.02 ROC-AUC, the largest drops. Removing the climate push, the attention or the gate changes accuracy by less than one standard deviation; these parts are kept because they provide the per-district explanation.

Table 5.6: Ablation of B-ACPN (ROC-AUC, 25 Folds)

Variant	ROC-AUC	Change vs full	p vs full
Full B-ACPN	0.820	—	—
Deep stage only (no tree stage)	0.800	-0.019	0.497
Tree stage only (no deep stage)	0.818	-0.002	0.952
Without climate push	0.816	-0.004	0.882
Without development gate	0.821	+0.001	0.959
Without feature attention	0.817	-0.003	0.926
Single network (no ensemble)	0.801	-0.018	0.579

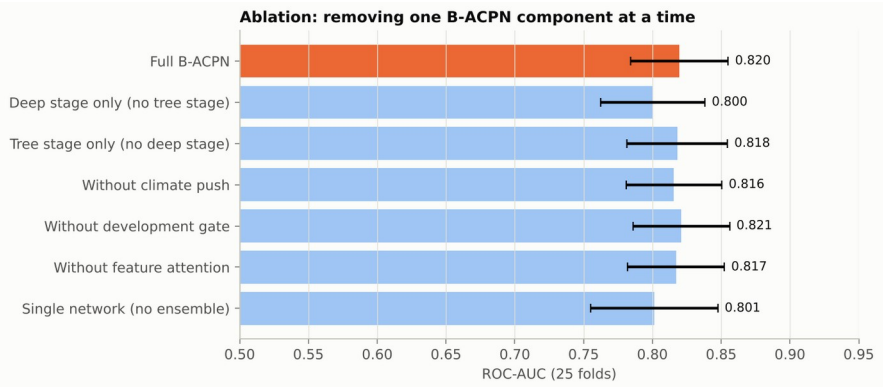


Fig. 5.6: Ablation: One B-ACPN Component Removed at a Time

5.5 Climate-Push Analysis

Table 5.7 shows the out-of-fold climate push per group. Developed but vulnerable districts (Anomaly A) have a positive mean push of +0.09, against -0.10 for developed districts that are not vulnerable (p = 1.6e-05). Less developed but resilient districts (Anomaly B) have a negative push of -0.17, against +0.13 for the expected group (p = 4.5e-07). Anomaly A districts have about twice as many tropical nights as expected developed districts (91 vs 45 per year). Attention gives heavy-rain days the largest weight (Fig. 5.7), and the Spark climate regimes also differ in push, which supports that the push follows real climate patterns. Fig. 5.8 maps the climate push of every district.

Table 5.7: Mean Climate Push by Group

Group	Districts	Mean push	Push > 0	Top factor
Developed, low vulnerability (expected)	143	-0.10	45%	heavy rain days
Less developed, high vulnerability (expected)	145	+0.13	86%	heavy rain days
Anomaly A: developed yet vulnerable	92	+0.09	82%	heavy rain days
Anomaly B: less developed yet resilient	89	-0.17	46%	heavy rain days

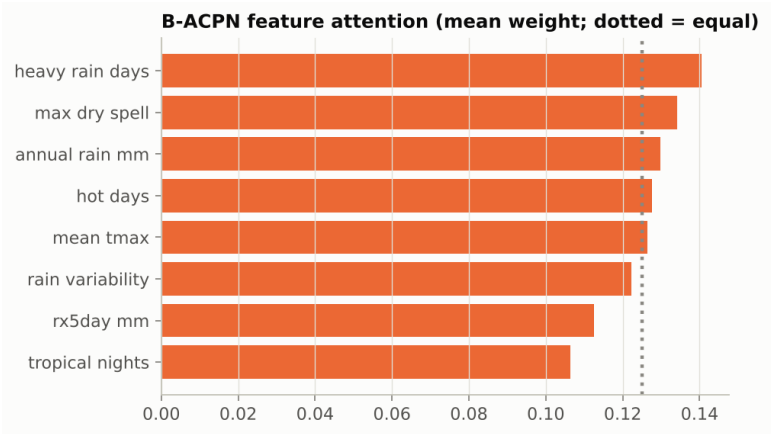


Fig. 5.7: Mean Feature-Attention Weight of Each Climate Variable

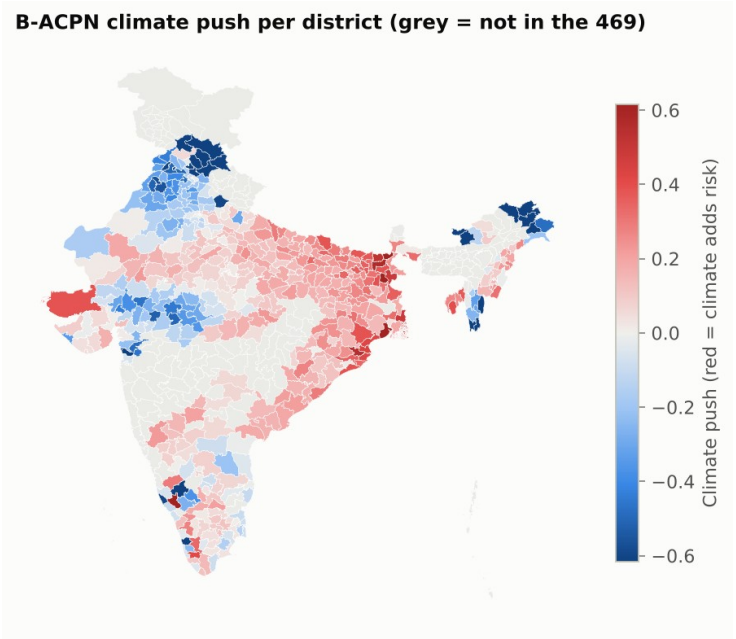


Fig. 5.8: Climate Push of Every District

5.6 Summary of Findings

- Development alone explains only 0.127 of the variance of vulnerability; adding the eight climate hazards raises it to 0.334.
- The Hadoop pipeline processes 1.08 GB, and every job matches an independent check.
- B-ACPN is the most accurate of six models (ROC-AUC 0.820) and comparable to GBT.
- Its climate push explains both anomaly groups with $p < 0.001$.
- Limitations: Census 2011 against a 2021 index, a 1° temperature grid, single-node Hadoop and 469 districts.

CHAPTER 6

CONCLUSION AND FUTURE WORK

This project combined a Hadoop pipeline over thirty years of daily IMD climate data with a novel hybrid model, B-ACPN, to explain why development does not always predict climate resilience in Indian districts. HDFS, MapReduce on YARN, Hive and Spark each performed a measured step, and preprocessing was verified with error metrics without imputing any value. B-ACPN matched or exceeded five existing models on every main metric and, unlike them, assigned each district a climate push with uncertainty. The climate push shows that climate explains much of why developed districts remain vulnerable and why some less-developed districts are resilient.

The work has clear limits. Development data come from Census 2011 while the index is from 2021; temperature is on a coarse 1° grid; only five development variables survive without imputation; Hadoop ran on a single node, so no multi-machine speed-up is claimed; and the lead of B-ACPN over gradient-boosted trees is not statistically significant.

Future work will extend HDFS to the full 1901–2024 rainfall record and re-time the pipeline, tune B-ACPN further, compare the district explanations with national vulnerability rankings [1], [2], and move the pipeline to a multi-node cluster. The same approach could be applied to other outcomes, such as crop losses or health impacts of heat.

REFERENCES

[1] A. Mohanty and S. Wadhawan, “Mapping India’s climate vulnerability: A district-level assessment,” Council on Energy, Environment and Water (CEEW), New Delhi, 2021.

[2] Department of Science and Technology, Government of India, “Climate vulnerability assessment for adaptation planning in India using a common framework,” New Delhi, 2020.

[3] IPCC, Climate Change 2022: Impacts, Adaptation and Vulnerability. Contribution of Working Group II to the Sixth Assessment Report. Cambridge, U.K.: Cambridge Univ. Press, 2022.

[4] D. S. Pai, L. Sridhar, M. Rajeevan, O. P. Sreejith, N. S. Satbhai and B. Mukhopadhyay, “Development of a new high spatial resolution (0.25° × 0.25°) long period (1901–2010) daily gridded rainfall data set over India and its comparison with existing data sets over the region,” MAUSAM, vol. 65, no. 1, pp. 1–18, 2014.

[5] A. K. Srivastava, M. Rajeevan and S. R. Kshirsagar, “Development of a high resolution daily gridded temperature data set (1969–2005) for the Indian region,” Atmospheric Science Letters, vol. 10, no. 4, pp. 249–254, 2009.

[6] K. Shvachko, H. Kuang, S. Radia and R. Chansler, “The Hadoop distributed file system,” in Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST), 2010, pp. 1–10.

[7] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” Commun. ACM, vol. 51, no. 1, pp. 107–113, 2008.

[8] V. K. Vavilapalli et al., “Apache Hadoop YARN: Yet another resource negotiator,” in Proc. 4th ACM Symp. Cloud Computing (SoCC), 2013, pp. 1–16.

[9] A. Thusoo et al., “Hive: A warehousing solution over a map-reduce framework,” Proc. VLDB Endowment, vol. 2, no. 2, pp. 1626–1629, 2009.

[10] M. Zaharia et al., “Apache Spark: A unified engine for big data processing,” Commun. ACM, vol. 59, no. 11, pp. 56–65, 2016.

[11] X. Meng et al., “MLlib: Machine learning in Apache Spark,” J. Mach. Learn. Res., vol. 17, no. 34, pp. 1–7, 2016.

[12] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[13] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[14] L. Grinsztajn, E. Oyallon and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” in Advances in Neural Information Processing Systems (Datasets and Benchmarks Track), vol. 35, 2022.

[15] S. Ö. Arık and T. Pfister, “TabNet: Attentive interpretable tabular learning,” in Proc. AAAI Conf. Artificial Intelligence, vol. 35, no. 8, 2021, pp. 6679–6687.

[16] D. H. Wolpert, “Stacked generalization,” Neural Networks, vol. 5, no. 2, pp. 241–259, 1992.

[17] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[18] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,” in Proc. 33rd Int. Conf. Machine Learning (ICML), 2016, pp. 1050–1059.

[19] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” arXiv:2107.07511, 2021.

[20] C. Nadeau and Y. Bengio, “Inference for the generalization error,” Machine Learning, vol. 52, no. 3, pp. 239–281, 2003.

[21] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” J. Comput. Appl. Math., vol. 20, pp. 53–65, 1987.